

Proyecto I: Illuminator

Juan José Rodríguez Chaves

Gabriel David Brenes Casasola

Escuela de ingeniería en Computadores, Instituto Tecnológico de Costa Rica

CE-1103: Algoritmos y Estructura de Datos

Leonardo Andres Araya Martinez

6 de enero de 2026

	2
1. Introducción	3
2. Descripción del Problema	4
3. Diseño General	5
3.1 Descripción de clases:	5
3.2 Diagrama del programa:	10

1. Introducción

El proyecto Illuminator consiste en un sistema de iluminación inteligente desarrollado con Arduino para un hogar, en este caso para la maqueta de una casa. El sistema integra distintos componentes electrónicos básicos como LEDs, pushbuttons, resistencias, protoboard, entre otras, a través de programación orientada a objetos (POO) que simulará el comportamiento de una casa inteligente.

El objetivo principal de nuestro proyecto es el de demostrar la aplicación de conceptos de programación orientada a objetos, polimorfismo y componentes electrónicos básicos para la implementación de un modelo de casa inteligente en un microcontrolador y así resolver un problema práctico de automatización residencial.

2. Descripción del Problema

En el contexto del proyecto se requiere de un sistema que pueda gestionar la iluminación de los distintos espacios de una vivienda de manera inteligente. El sistema de esta vivienda debe:

1. Detectar las condiciones ambientales a través de un sensor.
2. Reaccionar automáticamente ante estos.
3. Permitir el control manual de luces sobre cada espacio de la vivienda de manera individual.
4. Ofrecer distintos modos de iluminación predefinidos para distintas situaciones del día a día
5. Adicional a todo, ofrecer una retroalimentación visual del estado del sistema, (en este caso a través de una pantalla).

El problema del proyecto radica en integrar distintos componentes de hardware con un diseño de software modular y extensible que respete siempre las limitaciones de los recursos del microcontrolador.

3. Diseño General

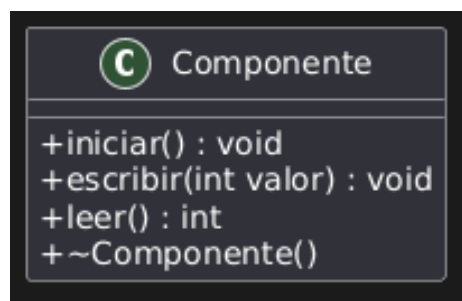
3.1 Descripción de clases:

La clase **Componente** es la clase madre, define una interfaz común para todos los componentes electrónicos: sensor Idr, LEDs, botones y pantallaLCD.

Posee solamente 4 métodos, todos se reescribirán en sus clases hijas y funcionan para la función esencial de

todo componente electrónico, encender, escribir o leer. Además, su ultimo método

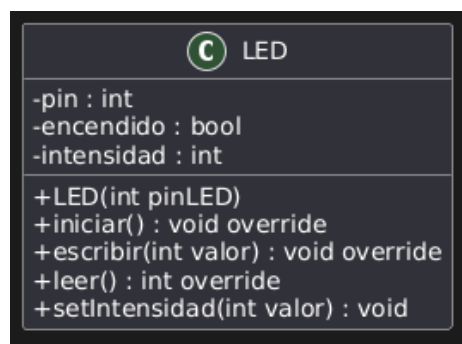
“~Componente()” es un destructor para eliminarse a si misma y ahorrar espacio en memoria.



La clase **LED** posee 3 atributos, el pin de entrada; siendo un int, encendido; siendo un booleano y la intensidad de la LED; siendo también un int. Todos estos funcionan para definir la cantidad de luz que emerge cada LED, dónde debe conectarse y que estado tiene: True para encendido, False para apagado.

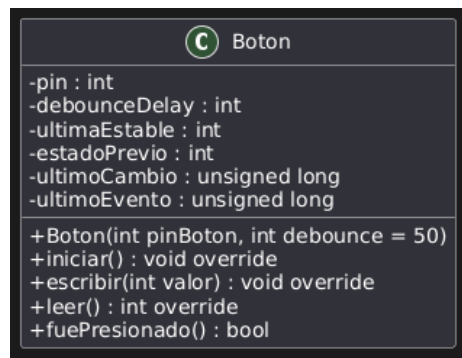
Luego, sus métodos se conforman de:

- `LED(int pinLED)`: Constructor que recibe el parámetro del pin.
- `Iniciar()`: Se encarga de iniciar en el pin establecido y leer como output.
- `Escribir()`: Este método busca encender la LED cuando el valor leído sea de 1, apagar la LED cuando el valor leído sea de 0 y establecer la intensidad de luz si el valor leído es mayor o igual a 2.
- `Leer()`: Retorna la lectura, encendido si 1, apagado si 0.
- `setIntensidad()`: Se encarga de regular la intensidad de luz para cada modo de luz, fijando el limite entre 0 y 255.



La clase **Boton** es un poco más extensa, posee varios atributos encargados de un simple pero a la vez complejo mecanismo de atributos

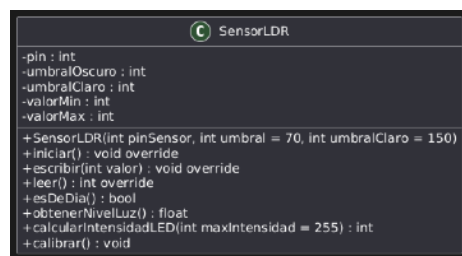
- Pin: Identifica el pin físico donde está conectado el pushbutton.
- DebouceDelay: Filtra los rebotes de los botones físicos, ya que no siempre pueden hacer un contacto limpio debido a la oscilación, por defecto se define en 50ms.
- ultimaEstable: Almacena el ultimo estado estable confirmado luego del debounce.
- estadoPrevio: Similar al atributo anterior almacena el estado leído anteriormente sin procesar para comparar lecturas nuevas y detectar cambios.
- ultimoCambio: Mide el tiempo que ha pasado desde el último cambio con una marca de tiempo en milisegundos.
- ultimoEvento: Funciona como una marca de tiempo del último evento registrado. Evita múltiples detecciones de una misma pulsación.
- Boton(int pinBoton, int debounce=50): Inicializa todos los atributos de la clase, o sea, es el constructor.
- Iniciar(): Lee el estado que se da justo después de iniciar y confirma el pin en "INPUT_PULLUP", esta activa la resistencia interna del Arduino sin necesidad de usar resistencias externas.
- Escribir(): Este es un método vacío pero necesario para el polimorfismo.
- Leer(): El método primero hace la lectura cruda, si la lectura no es igual a estadoPrevio guarda el tiempo actual en ultimoCambio y actualiza estadoPrevio. En caso de que el tiempo actual sea menor al debounceDelay desde ultimoCambio y de que la lectura sea diferente a ultimaEstable, se actualiza este ultimo y se retorna. De esta manera se lee el estado del botón de una manera más limpia.



- fuePresionado(): Similar a leer(), pero no igual, detecta el momento en el que se presiona el botón y devuelve true o false dependiendo de si paso de HIGH a LOW y entre eventos espera a ultimoEvento y a debounceDelay.

La clase **SensorLDR** es la encargada de leer la fotorresistencia.

- Pin: Identifica el pin físico en el que esta conectada la fotorresistencia.
- umbralOscuro: Determina un valor analógico en el cual se establece el punto en el que se considera que hay poca luz.
- umbralClaro: Determina un valor analógico en el cual se establece el punto en el que se considera que hay mucha luz.
- valorMin: Valor mínimo registrado para la calibración dinámica del sensor.
- valorMax: Valor máximo registrado para la calibración dinámica del sensor.
- SensorLDR(...): Constructor; (valorMin se establece por defecto en 512 y valorMax en 0).
- iniciar(): Inicia el pin como entrada analógica.
- escribir(): No aplica para el sensor pero es necesario para el polimorfismo.
- leer(): Retorna el valor del sensor LDR.
- esDeDia(): Es un método booleano que registra si la luz es suficiente para apagar los LEDs.
- obtenerNivelLuz(): Normaliza la lectura del sensor entre 0.0 y 1.0
- calcularIntensidadLED(): Calcula la intensidad PWM de los LEDs basándoselas en la luz ambiental.
- Calibrar(): Calibra el sensor tomando los umbrales y reinicia los valores.



```

class SensorLDR {
  -pin : int
  -umbralOscuro : int
  -umbralClaro : int
  -valorMin : int
  -valorMax : int

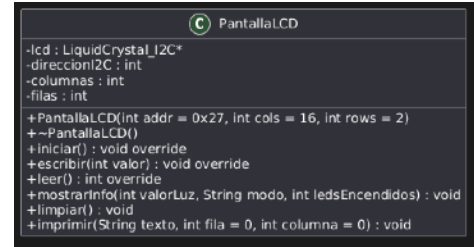
  +SensorLDR(int pinSensor, int umbral = 70, int umbralClaro = 150)
  +iniciar() : void override
  +escribir(int valor) : void override
  +leer() : int override
  +esDeDia() : bool
  +obtenerNivelLuz() : float
  +calcularIntensidadLED(int maxIntensidad = 255) : int
  +calibrar() : void

```

La clase **PantallaLCD** es la encargada de la pantalla que muestra la información vital de la casa y posee los siguientes atributos y métodos.

- LiquidCrystal_I2C*: Se trata de una abstracción en la que utilizamos la biblioteca ya establecida y creamos el constructor a base de esta.

- direccionI2C: Esta es la dirección I2C del modulo LCD, por defecto la dirección está establecida como 0x27 y así identificamos el dispositivo en el bus I2C.
 - Columnas: Para la pantalla utilizada en el proyecto se necesitan establecer 16 columnas, ya que estamos trabajando con una pantalla 1602.
 - Filas: Al igual que las columnas y para el manejo de la visualización, se establecen 2 filas para organizar el contenido.
- PantallaLCD(int addr=0x27, int cols=16, int rows=2): Constructor de la clase, crea e inicializa el objeto.
 - ~PantallaLCD(): Libera la memoria dinamica de lcd.
 - iniciar(): Inicializa la comunicación I2C con la pantalla y enciende todos los mecanismos con la luz de fondo, con clear() limpia el contenido y se establece un print para demostrar que el sistema ha iniciado con éxito.
 - escribir(): No tiene uso pero es necesario para el polimorfismo.
 - leer(): No tiene uso pero es necesario para el polimorfismo.
 - mostrarInfo(int valorLuz, String modo, int ledsEncendidos): La clase no necesita de los métodos anteriores ya que este método funciona de manera perfecta para mostrar toda la información requerida por las indicaciones del proyecto, enseña el valor que detecta la fotorresistencia, el modo de luces y la cantidad de LEDs encendidas a través de la librería mencionada anteriormente.
 - limpiar(): Método simple que limpia y borra toda la pantalla para reiniciarla.
 - imprimir(String texto, int fila=0, int columna=0): Similar a mostrar info, este método muestra distintos mensajes en la pantalla a conveniencia. Es mejor tener dos métodos que, aunque hagan funciones similares, puedan realizar distintos mensajes sin entrar en conflictos.



```

class PantallaLCD {
    -lcd : LiquidCrystal_I2C*
    -direccionI2C : int
    -columnas : int
    -filas : int

    +PantallaLCD(int addr = 0x27, int cols = 16, int rows = 2)
    +~PantallaLCD()
    +iniciar() : void override
    +escribir(int valor) : void override
    +leer() : int override
    +mostrarInfo(int valorLuz, String modo, int ledsEncendidos) : void
    +limpiar() : void
    +imprimir(String texto, int fila = 0, int columna = 0) : void
  
```


Así, el proyecto se “une” en la clase “Casa”, el cerebro del sistema y la clase principal que coordina todos los componentes y contiene toda la lógica que simplifica el uso de un sistema tan complejo como este y aunque no herede nada agrega todos los demás componentes. En síntesis, tiene grandes responsabilidades:

- Gestionar el hardware, inicializar todos los componentes físicos y gestionar sus ciclos de memoria.
- Coordinar los distintos modos de 7 y cambiar entre estos.
- Implementar el control combinado de los botones para el modo manual y el modo automático que detecta la luz ambiental.

```

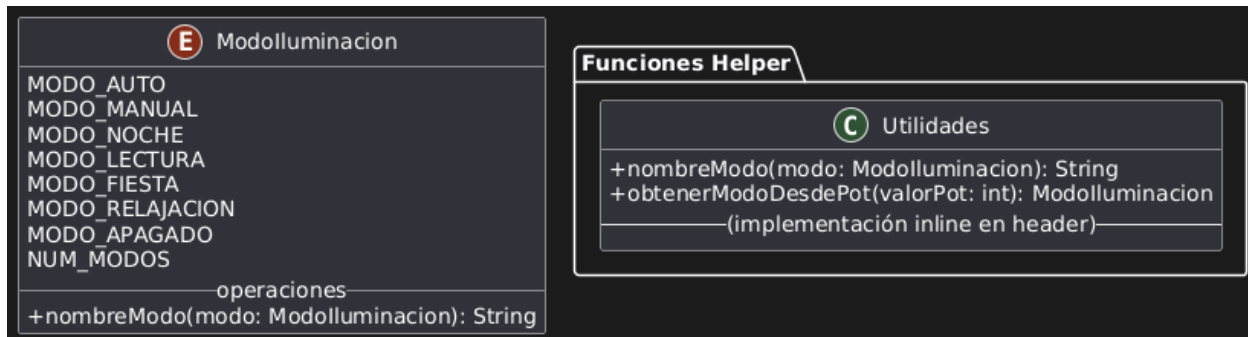
C Casa

-sala : LED*
-cocina : LED*
-cuarto1 : LED*
-patioInterno : LED*
-patioFrontal : LED*
-patioTrasero : LED*
-sensor : SensorLDR*
-botonSala : Boton*
-botonCocina : Boton*
-botonCuarto1 : Boton*
-botonPatioInt : Boton*
-botonPatioFront : Boton*
-botonPatioTras : Boton*
-PIN_POTENCIOMETRO = A1: static const int
-pantalla : PantallaLCD*
-modoActual : Modolluminacion
-modoAnterior : Modolluminacion
-ledsEncendidos : int
-ultimaActualizacionPot: unsigned long
-ultimaActualizacionLCD: unsigned long
-estadoManual[6] : bool
-overrideManual[6] : bool
-intensidadAutoArray[6] : int
-UMBRAL_OSCURO = 50: static const int
-UMBRAL_CLARO = 200: static const int

+ Casa()
+ iniciar() : void
+ actualizar() : void
+ cambiarModo(Modolluminacion nuevoModo) : void
+ contarLEDsEncendidos() : int
+ aplicarModoGlobal(Modolluminacion modo) : void
+ toggleLEDManual(int index) : void
+ aplicarControlCombinado() : void
-actualizarLCD() : void
-verificarBotones() : void
-controlAutomatico() : void
-leerPotenciometro() : void
-obtenerModoDesdePot(int valorPot) : Modolluminacion

```

Así, la clase a través de su numerosa cantidad de atributos define todos los componentes físicos y distintas características necesarias para el correcto funcionamiento del mecanismo. El método más importante de la clase es actualizar(), pues se trata del “corazón” de la clase, aquella que se encargará del loop principal.



Luego encontramos otras clases más sencillas que simplemente funcionan para almacenar información y leer el potenciómetro de una manera más exacta y sin problemas.

3.2 Diagrama del programa:

